

Módulo de agendamento de consultas

Stack: React Native com Expo (JavaScript) + API em PHP 8.2+ e Laravel compatível

Entrega: repositório Git público ou privado, com acesso concedido aos avaliadores

1 Objetivo

Desenvolver uma aplicação mobile integrada a uma API REST para simular o agendamento e a consulta do histórico de atendimentos de um paciente.

O teste busca avaliar:

- organização e clareza do código;
- integração entre aplicativo e API;
- implementação de regras de negócio;
- qualidade da experiência no aplicativo;
- capacidade de documentar e justificar decisões técnicas.

O escopo foi pensado para caber em até 12 horas. Caso não seja possível concluir tudo, priorize um fluxo funcional e explique no README o que ficou pendente e como seria finalizado.

2 Entrega esperada

O repositório deve conter:

```
backend/  
mobile/  
README.md
```

O candidato deve entregar:

1. uma API REST em Laravel;
2. um aplicativo em React Native com Expo;
3. migrations e seeders para preparar o banco de dados;
4. um README com instruções para executar os projetos e um resumo das decisões técnicas.

Não é necessário implementar cadastro ou autenticação. A aplicação utilizará um paciente de demonstração criado pelo seeder. O identificador desse paciente pode ser configurado no aplicativo ou informado nas requisições, desde que a solução esteja documentada.

3 Escopo funcional

3.1 Agendar consulta

O aplicativo deve permitir:

5. selecionar uma especialidade;
6. selecionar um profissional daquela especialidade;
7. escolher uma data e um horário;
8. revisar os dados;
9. confirmar o agendamento.

3.2 Consultar histórico

O aplicativo deve exibir as consultas do paciente de demonstração, contendo ao menos:

- especialidade;
- profissional;
- data e horário;
- status.

A lista deve permitir filtro por status e uma ação para atualizar os dados.

3.3 Consultar e cancelar uma consulta

O candidato deve disponibilizar uma visualização dos detalhes da consulta. Consultas com status agendado podem ser canceladas.

4 Regras de negócio

Código	Regra
RN-01	Não permitir agendamento em data ou horário passado.
RN-02	Um profissional não pode possuir duas consultas no mesmo horário.
RN-03	Todo agendamento deve estar vinculado ao paciente de demonstração e a um profissional.
RN-04	O status inicial de uma consulta deve ser agendado.
RN-05	Os status aceitos são agendado, confirmado, realizado e cancelado.
RN-06	Somente consultas com status agendado ou confirmado podem ser canceladas.

As validações devem ocorrer no backend. O aplicativo deve apresentar mensagens compreensíveis quando uma operação não puder ser concluída.

5 API mínima

As rotas abaixo representam o contrato mínimo. Ajustes de nomenclatura são permitidos quando documentados no README.

Método	Rota	Finalidade
GET	/api/specialties	Listar especialidades.
GET	/api/professionals?specialty_id={id}	Listar profissionais de uma especialidade.
GET	/api/appointments?patient_id={id}&status={status}	Listar consultas do paciente e, opcionalmente, filtrar por status.
POST	/api/appointments	Criar um agendamento.
GET	/api/appointments/{id}	Consultar os detalhes de um agendamento.
PATCH	/api/appointments/{id}/cancel	Cancelar um agendamento permitido pela RN-06.

As respostas de sucesso e erro devem usar JSON e códigos HTTP adequados. Não é necessário criar um formato global sofisticado de erros.

6 Modelo de dados

O modelo deve contemplar, no mínimo, as entidades abaixo. Tipos, índices e detalhes de implementação ficam a critério do candidato.

Entidade	Campos essenciais
patients	identificador e nome
specialties	identificador e nome
professionals	identificador, especialidade e nome
appointments	identificador, paciente, profissional, data e horário, status e observações opcionais

Relacionamentos esperados:

- uma especialidade possui vários profissionais;
- um profissional pertence a uma especialidade e possui várias consultas;
- um paciente possui várias consultas;
- uma consulta pertence a um paciente e a um profissional.

6.1 Dados mínimos de seed

- 3 especialidades;
- 6 profissionais, distribuídos entre as especialidades;
- 1 paciente de demonstração;
- 4 consultas de exemplo, contemplando diferentes status.

7 Requisitos do aplicativo

O aplicativo deve incluir:

- tela inicial com acesso ao agendamento e ao histórico;
- fluxo de agendamento;
- histórico com filtro por status e atualização manual;
- detalhes e cancelamento da consulta;
- estados visíveis de carregamento, erro e conteúdo vazio;
- validação básica dos dados informados;
- navegação organizada.

O projeto deve usar React Native com Expo e JavaScript. A escolha de biblioteca HTTP, formulários, estado e navegação é livre e deve ser brevemente explicada no README.

Não há estrutura de pastas obrigatória. Espera-se apenas uma separação clara de responsabilidades e que as telas não concentrem indevidamente regras de negócio ou detalhes de comunicação com a API.

8 Requisitos do backend

O backend deve apresentar:

- rotas REST organizadas;
- validação das entradas;
- regras de negócio separadas da camada de transporte HTTP;
- modelos e relacionamentos consistentes;
- migrations e seeders executáveis;
- respostas JSON claras;
- tratamento adequado das falhas previstas no escopo.

9 README e execução

O README deve informar:

- pré-requisitos;
- configuração das variáveis de ambiente;
- instalação das dependências;
- criação e preparação do banco de dados;
- execução da API e do aplicativo;
- identificação do paciente de demonstração;
- decisões técnicas relevantes;
- funcionalidades concluídas e eventuais pendências.

O arquivo `.env.example` do backend deve conter todas as variáveis necessárias, sem segredos ou credenciais reais.

Soluções simples, consistentes e bem explicadas serão melhor avaliadas do que implementações extensas e incompletas.